

MID SEMESTER EXAMINATION - 1, Blended ODD SEM - 2026-27

CS336B - Object Oriented Programming with Java

Programme Name: B.Tech CS, CSAI,
CSAIML, CSE, CSE-CS, CSE-DS, CSIT,
IT

Semester: III

Max. Marks: 40

Max. Time: 2 hours

Question Paper ID: 002311

Instructions:

All questions are compulsory.

Assume suitable data wherever necessary and mention it clearly.

Attempt the Question (1 × 5 =
5 Marks)

Marks CO BL

- Q1.** A space mission system calculates the required fuel for a spacecraft. The required fuel is calculated as: 5 1 3
- Required Fuel = Distance / Fuel Efficiency
- (a) Create a class Spacecraft with distance and fuelEfficiency as data members.
- (b) Create a method calculateFuel() that calculates the required fuel. If fuelEfficiency is 0, handle the exception using try-catch and display message "Invalid fuel efficiency!".

Driver code:

```
public class Main{
    public static void main(String[] args) {
        Spacecraft s = new Spacecraft();
        s.distance=10;
        s.fuelEfficiency=0;
        s.calculateFuel();
    }
}
```

```
class Spacecraft{
    //Write your code here
}
```

Attempt the Question (1 × 5 =
5 Marks)

Marks CO BL

- Q2.** A museum maintains a list of rare artifacts currently displayed in an exhibition. New artifacts can be added, and an artifact may be removed when it is sent for restoration. The museum uses a LinkedList to manage this dynamic list. 5 2 3
- (a) Create a LinkedList to store artifact names.
- (b) Add three artifact names to the list.
- (c) Add one new artifact at the beginning of the list.
- (d) Remove one artifact from the list and display the updated list.
- (e) Display the first and last artifact in the list.

Driver code:

```
import java.util.LinkedList;
class ArtifactTracker {
    public static void main(String[] args) {
        //Write your code here
    }
}
```

- Q3.** Smart Parking System manages a limited number of parking slots. Two vehicles try to book a parking slot at the same time. Since both vehicles access the same availableSlots variable, synchronization is required to avoid incorrect slot booking. (a) Create a class ParkingArea that extends Thread. Declare a variable vehicleNo and a shared variable availableSlots. Create a static synchronized method parkVehicle(String vehicleNo) that books a slot if available and decreases availableSlots. (b) Override the run() method to call parkVehicle().

Driver code:

```
public class Main{
    public static void main(String[] args) {
        ParkingArea v1=new ParkingArea();
        v1.vehicleNo="UP11K2026";
        v1.start();

        ParkingArea v2=new ParkingArea();
        v2.vehicleNo="Uk08K2020";
        v2.start();
    }
}
```

```
class ParkingArea extends Thread{
    //Write your code here
}
```

- Q4.** A smart EV charging station wants to develop a system to keep track of electric vehicles using its charging facility. Each vehicle has a registration number, vehicle model and battery percentage. The station wants to prevent direct access to vehicle information, maintain the total number of vehicles registered, and ensure that every vehicle has a unique registration number. (a) Create a class EVVehicle and declare registrationNo, model and batteryPercentage as private data members. Provide appropriate getter and setter methods to ensure controlled access to the data. (b) Create a parameterized constructor to initialize all vehicle details when an object is created. (c) Declare a static variable vehicleCount that keeps track of the total number of EVVehicle objects created. (d) Declare a final variable stationName and initialize it with "GreenCharge EV Station".

Driver Code:

```
public class Main{
    public static void main(String[] args) {
        EVVehicle v1=new EVVehicle(1,"Nexon",78);
        EVVehicle e2=new EVVehicle(2,"Brezza",50);

        System.out.println("Station: " + EVVehicle.stationName);
        System.out.println(v1.getRegistrationNo() + " " + v1.getModel() + " " + v1.getBatteryPercentage());
        System.out.println("Total Vehicles: " + EVVehicle.vehicleCount);
    }
}
```

```

    }
}
class EVVehicle{
    //Write your code here
}

```

Q5. A Smart Home System uses a smart device that can perform both security monitoring and energy monitoring. 7.5 1 3

- (a) Create an interface SecuritySystem with a method monitorSecurity().
 (b) Create another interface EnergySystem with a method checkEnergyUsage().
 (c) Create a class SmartHome that implements both SecuritySystem and EnergySystem. Override both methods with a customize message as follows:
 "Security is monitored"
 "Energy check done"

Driver code:

```

public class Main {
    public static void main(String[] args) {
        SmartHome sh = new SmartHome();
        sh.monitorSecurity();
        sh.checkEnergyUsage();
    }
}

```

//Write interfaces and class code here.

**Attempt any two questions (2
 × 7.5 = 15 Marks)**

Marks CO BL

Q6. An Online Shopping System maintains a list of products available for purchase. Each product has a product ID, product name, and price. The system uses an ArrayList to store and manage product details. 7.5 2 3

- (a) Create a class ProductDetails with data members productId, productName, and price and make a constructor to initialize all variables.
 (b) In the main() method, create an ArrayList of ProductDetails objects and add details of all three given products using the add() method.
 (c) Display all the products stored in the ArrayList using for each loop.
 (d) Use the remove() method to remove one product from the ArrayList and display the updated list.

Driver code:

```

Class ProductDetails{
    //write code here
}

```

```

public class Main{
    public static void main(String[] args) {
        ProductDetails p1=new ProductDetails(1,"Soap",100);
        ProductDetails p2=new ProductDetails(2,"Gel",300);
        ProductDetails p3=new ProductDetails(3,"Shampoo",400);
        //Write ArrayList code here
    }
}

```

- Q7.** A College Event Registration System maintains the names of students registered for a technical event. A student should be registered only once. 7.5 2 3
- (a) Select a suitable Java Collection to store the names of registered students such that duplicate names are not allowed.
- (b) Add at least four students to the collection.
- (c) Display the total number of registered students using the size() method.
- (d) Remove one student from the collection.
- (e) Write a suitable loop to display all the registered student names.

Driver code:

```
import java.util.HashSet;
public class Main{
    public static void main(String[] args) {
        //Write your code here
    }
}
```

- Q8.** A Food Order Management System maintains a list of food items selected by customers. New food items can be added at the beginning of the order list, while the last food item can be removed when required. 7.5 2 3
- (a) Create a LinkedList to store the names of food items.
- (b) Add the following food items at the beginning of the list:
- "Pizza"
 - "Burger"
 - "Pasta"
 - "Sandwich"
- (c) Display the first item of the list.
- (d) Remove one food item from the end of the list.
- (e) Write a suitable loop to display all the food items currently present in the list.

Driver code:

```
public class FoodOrder{
    public static void main(String[] args) {
        //write your code
    }
}
```

* * * * *